

4.1.2 Hasil *Preprocessing Data*

Setelah data paket wisata berhasil dikumpulkan, langkah selanjutnya adalah melakukan tahap *preprocessing* terhadap data teks. *Preprocessing* bertujuan untuk membersihkan data dari elemen-elemen yang tidak diperlukan seperti simbol, tag HTML, karakter khusus, serta kata-kata yang tidak memiliki nilai informatif. Proses ini dilakukan menggunakan beberapa *library* Python, yaitu *re (regular expression)* untuk membersihkan karakter non-alfabet, serta *PySastrawi* yang menyediakan *StopWordRemoverFactory* untuk penghapusan *stopword* dan *StemmerFactory* untuk proses *stemming* berbasis algoritma Nazief & Adriani.

1. *Case Folding*

Case Folding merupakan proses mengubah seluruh kalimat yang dalam bentuk huruf besar menjadi huruf kecil. Hal ini dilakukan dengan tujuan agar menyamakan representasi kata, sehingga kata seperti “Wisata”, “WISATA”, atau “wisata” dianggap sama dalam proses pencocokan dan perhitungan kemiripan.

Tabel 4.4 Proses *Case Folding*

```
def case_folding(text):
    if not text:
        return ""

    text = text.lower()
    text = re.sub(r'^a-zA-Z0-9\s', '', text)
    text = re.sub(r'\s+', '', text)

    return text.strip()
```

Pada Tabel 4.4 merupakan kode dari proses *Case Folding* yang digunakan untuk mengubah huruf besar (kapital) ke huruf kecil (lowercase) *.lower()*. dan memanfaatkan *Regular Expression re.sub* untuk menyaring teks sehingga hanya menyisakan huruf alfabet dan angka saja (menghapus titik, koma dll).

Tabel 4.5 Hasil *Case Folding*

Sebelum Diproses	Hasil Case Folding

penjelasan

2. *Stopword Removal*

Stopword removal pada penelitian ini mengacu pada daftar stopwords Bahasa Indonesia yang telah disesuaikan dengan kebutuhan sistem rekomendasi paket wisata. Proses ini bertujuan untuk menghilangkan kata-kata umum yang tidak memiliki kontribusi signifikan terhadap makna dokumen, seperti kata hubung, kata depan, dan kata keterangan tertentu. Dengan menghapus kata-kata tersebut, proses pembobotan TF-IDF dapat lebih fokus pada kata-kata yang merepresentasikan karakteristik paket wisata. Daftar lengkap *stopword* yang digunakan dalam penelitian ini dapat dilihat pada Lampiran 4 List *Stopword*.

Tabel 4.6 Proses *Stopword Removal*

```
def remove_stopwords(text):  
    if not text:  
        return ""  
  
    words = text.split()  
    filtered_words = [word for word in words if word not in all_stopwords]  
    return " ".join(filtered_words)
```

Sistem memecah teks menjadi susunan kata, kemudian menyeleksi kata-kata tersebut. Jika kata tersebut terdaftar dalam kamus *all_stopwords* (yang berisi ratusan kata hubung bawaan *PySastrawi* serta *stopword* kustom pariwisata), maka kata tersebut otomatis akan dibuang.

Tabel 4.7 Hasil *Stopword Removal*

Sebelum Diproses	Hasil Stopword Removal

3. *Stemming*

Tahapan *Stemming* sangat krusial dalam proses pra pemrosesan data. Proses ini dilakukan untuk menyederhanakan kata-kata hasil filtering menjadi bentuk dasarnya atau akar katanya menggunakan algoritma Nazief & Adriani dari library *PySastrawi*. Tahapan ini sangat penting untuk menghindari pengulangan makna yang berasal dari bentuk kata berimbuhan yang berbeda namun memiliki arti akar yang sama.

Tabel 4.8 Proses *Stemming*

```
def stemming(text):
    if not text:
        return ""
    try:
        words = text.split()
        stemmed_words = []
        for word in words:
            # Pengecualian agar nama lokasi destinasi tidak ter-stemming
            if word in stemming_exemptions:
                stemmed_words.append(word)
            else:
                stemmed_words.append(stemmer.stem(word))
        return " ".join(stemmed_words)
    except Exception as e:
        return text
```

Fungsi pada Tabel 4.6 memanggil objek *stemmer.stem()* untuk menghilangkan awalan maupun akhiran kata. Kode ini juga mengimplementasikan teknik pengecualian khusus (*stemming_exemptions*) untuk nama-nama destinasi wisata (*Named Entity*) agar tidak rusak akibat proses pemotongan imbuhan yang berlebih.

Tabel 4.9 Hasil *Stemming*

Sebelum Diproses	Hasil <i>Stemming</i>

4. *Tokenizing*

Tokenizing merupakan suatu proses dimana teks akhir dipotong menjadi token-token (kata) terpisah dan dibersihkan dari nilai karakter tunggal yang tidak memiliki arti pembobotan (seperti angka lepasan), sehingga diperoleh daftar token bersih dari setiap paket wisata.

Tabel 4.10 Proses *Tokenizing*

```
def tokenize(text):
    if not text:
        return ""
    words = text.split()
```

```
filtered_tokens = [word for word in words if len(word) >= 2]
return " ".join(filtered_tokens)
```

Fungsi pada Tabel 4.8 yaitu sistem memecah ulang kalimat yang sudah melalui proses *stemming*, kemudian melakukan perhitungan panjang karakter menggunakan fungsi *len()*. Apabila suatu kata atau angka hanya terdiri dari satu karakter, maka token tersebut otomatis dibuang dari daftar final.

Tabel 4.11 Hasil *Tokenizing*

Sebelum Diproses	Hasil <i>Stemming</i>

Penjelasan singkat

4.1.3 Hasil TF-IDF

Setelah semua data deskripsi paket wisata melalui tahapan *pre-processing* (yang mencakup *case folding*, *stopword removal*, *stemming* menggunakan algoritma *PySastrawi*, dan *tokenizing*), diperoleh kumpulan kata unik (vocabulary) yang membentuk representasi fitur dari koleksi dokumen database. Kata-kata yang telah dibersihkan ini digunakan sebagai dasar dalam membangun representasi numerik dokumen menggunakan teknik pembobotan TF-IDF (*Term Frequency - Inverse Document Frequency*). Pembobotan TF-IDF bertujuan untuk menghitung seberapa penting atau relevan sebuah kata terhadap keseluruhan koleksi dokumen paket wisata. Sistem pada penelitian ini mengimplementasikan library *Scikit-Learn* (*TfidfVectorizer*) yang secara otomatis memproses nilai TF dan IDF secara serentak.

Menghitung seberapa sering sebuah kata (token) muncul di dalam satu dokumen paket wisata. Semakin sering kata tersebut disebut (akibat dari teknik *Feature Engineering* pengulangan kalimat deskripsi), maka bobot kemunculannya semakin tinggi.

Tabel 4.12 Proses *TF-IDF*

```
vectorizer = TfidfVectorizer(  
    analyzer='word',  
    ngram_range=(1, 2),  
    min_df=1,  
    sublinear_tf=True  
)  
tfidf_matrix = vectorizer.fit_transform(corpus_list)
```

Pada Tabel 4.10 menjalankan algoritma TF-IDF. Parameter *ngram_range*=(1, 2) diatur agar sistem mampu membaca kombinasi 2 kata berurutan (Bigram) seperti "api biru". Sedangkan parameter *sublinear_tf*=True diterapkan untuk menormalkan lonjakan skala pembobotan secara logaritmik akibat teknik *Feature Engineering* (pengulangan kalimat deskripsi). Proses perhitungan matematis dijalankan secara komprehensif dan simultan melalui fungsi bawaan *.fit_transform()*. Hasil Eksekusi Pembobotan (Nilai TF dan Nilai IDF), walaupun diproses secara serentak di dalam satu baris kode, algoritma ini secara otomatis membedah teks berdasarkan dua rumus matematis.

TINGGAL MASUKKAN HASIL DARI TF-IDF NYA

4.1.4 Hasil *Cosine Similarity*

Setelah proses pembobotan TF-IDF menghasilkan representasi matriks numerik untuk setiap paket wisata di database, langkah selanjutnya adalah mengukur tingkat kemiripan antara preferensi yang diketikkan pengguna dengan seluruh paket wisata yang tersedia. Pengukuran ini dilakukan menggunakan metode *Cosine Similarity*. *Cosine Similarity* bekerja dengan cara merepresentasikan teks input pengguna dan teks deskripsi paket wisata sebagai dua buah vektor dalam ruang multidimensi. Algoritma kemudian menghitung nilai cosine dari sudut (angle) yang terbentuk di antara kedua vektor tersebut. Hasil dari perhitungan ini digunakan sebagai dasar pijakan mutlak (core engine) bagi sistem dalam menentukan peringkat (ranking) rekomendasi, di mana paket wisata yang memiliki vektor dengan arah paling selaras (sudut terkecil) dengan niat pengguna akan ditampilkan pada urutan teratas.

Table 4.13 Proses *Cosine Similarity*

```
def calculate_similarity(preference_vector, package_vectors_list):

    pkg_matrix = np.array(package_vectors_list)
    pref_vector_2d = preference_vector.reshape(1, -1)
    similarity_matrix = cosine_similarity(pref_vector_2d, pkg_matrix)
    scores = similarity_matrix[0].tolist()

    return scores
```

Berbeda dengan beberapa pendekatan konvensional yang menghitung kemiripan menggunakan rumus matematis manual baris-per-baris, sistem pada penelitian ini dirancang untuk menangani komputasi tingkat lanjut dengan memanfaatkan pustaka *scikit-learn* (`sklearn.metrics.pairwise.cosine_similarity`). Fungsi pada Tabel 4.13 menerima matriks vektor preferensi pengguna (`pref_vector_2d`) dan membandingkannya sekaligus dengan seluruh kumpulan matriks vektor paket wisata dari database (`pkg_matrix`). Hasil akhir dari fungsi ini berupa nilai *cosine similarity* yang berada dalam rentang skor 0 hingga 1. Nilai yang semakin mendekati 1 menunjukkan tingkat kemiripan yang sangat tinggi (vektor saling berhimpit), sedangkan nilai yang mendekati 0 menunjukkan tidak adanya relevansi kata sama sekali (vektor tegak lurus).

TINGGAL MASUKKIN HASIL DAN PENJELASN SINGKAT